

Оператор return, аргументы и параметры функций

Сегодняшняя тема - это «сердце» программирования на Python. Если функции без параметров - это простые команды, то функции с аргументами и оператором return - это полноценные инструменты, которые принимают данные, обрабатывают их и выдают результат.



1. Параметры и Аргументы: Входные данные

Многие путают эти два понятия, но различие простое:

- **Параметры** - это переменные, которые мы указываем в скобках при создании функции (заглушки).
- **Аргументы** - это реальные значения, которые мы передаем в функцию при её вызове.

Синтаксис:

```
def name(parameter1, parameter2): # Параметры
    # код
    pass

name(value1, value2) # Аргументы
```

Пример:

```
def multiply(a, b): # a и b - параметры
    print(a * b)

multiply(10, 5) # 10 и 5 - аргументы
```



2. Оператор return: Выходные данные

Функция может не просто печатать что-то на экран (print), а возвращать результат своей работы в основную программу. Это позволяет сохранять результат в переменные и использовать его дальше.

Важно: Как только Python встречает return, **выполнение функции немедленно прекращается**. Всё, что написано после return внутри функции, не выполнится.

Сравнение Print vs Return:

Характеристика	print()	return
Действие	Показывает текст пользователю.	Отдает данные программе.
Дальнейшее использование	Невозможно сохранить результат.	Можно записать в переменную.
Завершение функции	Функция продолжает работу.	Функция сразу завершается.

3. Шаблоны и сложные примеры

Шаблон 1: Базовая математическая функция

```
def calculate_area(width, height):
    area = width * height
    return area

# Используем результат
my_room = calculate_area(5, 4)
print(f"Площадь комнаты: {my_room} м²")
```

Шаблон 2: Функция с условием

```
def check_age(age):
    if age >= 18:
        return "Доступ разрешен"
    else:
        return "Доступ запрещен"

status = check_age(20)
print(status)
```

Шаблон 3: Возврат нескольких значений

Python позволяет вернуть сразу несколько значений в виде кортежа.

```
def get_user_data():
    name = "Адиль"
```

```
age = 25
return name, age # Возвращает кортеж (name, age)

user_name, user_age = get_user_data() # Распаковка
print(user_name)
```

Примеры использования на практике

1. Калькуляторы: Функция принимает два числа и возвращает их сумму, разность и т.д.
2. Обработка данных: Функция получает сырой текст, очищает его и возвращает «чистую» строку.
3. Валидация: Функция проверяет правильность ИИН или пароля и возвращает True или False.

Задачи для закрепления

Задача 1: «Конвертер валют»

Напишите функцию `to_tenge(dollars, course)`, которая принимает количество долларов и текущий курс. Функция должна возвращать итоговую сумму в тенге. Выведите результат на экран.

Задача 2: «Поиск максимума»

Напишите функцию `find_max(a, b)`, которая принимает два числа и возвращает большее из них. Не используйте встроенную функцию `max()`, примените `if-else`.

Задача 3: «Валидатор пароля»

Напишите функцию `is_strong(password)`, которая принимает строку. Если длина строки больше 8 символов, функция возвращает `True`, иначе - `False`. Используйте результат в условии `if` вне функции.

Задача 4: «Среднее арифметическое»

Напишите функцию `get_average(numbers)`, которая принимает список чисел и возвращает их среднее значение.

Подсказка: используйте `sum(numbers) / len(numbers)`.

Задача 5: «Форматирование имени»

Напишите функцию `format_name(first, last)`, которая принимает имя и фамилию. Она должна возвращать одну строку, где оба слова написаны с заглавной буквы (используйте метод `.capitalize()`).

Пример: "иван", "иванов" -> "Иван Иванов".



Итог темы:

- Параметры создают гибкость (функция работает с разными данными).
- Return делает функцию полезной для других частей кода (результат можно передавать по цепочке).
- Функция без return всегда неявно возвращает None.